ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ІВАНА ФРАНКА
Факультет електроніки та комп'ютерних технологій

ЗВІТ
ДО ЛАБОРАТОРНОЇ РОБОТИ №10

«Production Python: Docker, CI/CD та захист проєкту»

з курсу «Професійний Python»
Варіант №13 — «Система обліку комп'ютерної техніки»

Виконав: Капарник Дмитро Андрійович

Група: ФЕП-32

Перевірив: доцент Куньо Іван Михайлович

Дата: 18.10.2026

Львів — 2026

1. Мета роботи

Підготувати відтворюваний Python distribution, конфігурацію середовища, контейнерний запуск і автоматичні перевірки для фінальної демонстрації.

2. Індивідуальний варіант і постановка задачі

Варіант №13 — система обліку комп'ютерної техніки. Фінальний package/module — inventory-service / inventory_service. Зберегти SQLite, REST API, предметні функції та доречні оптимізації. Створити wheel/sdist, перевірити wheel у чистому venv, підготувати Docker і GitHub Actions.

Наскрізний проєкт продовжує ЛР №1. Базові поля: інвентарний номер, тип, виробник, модель і вартість. Збережено пошук, фільтр виробника, середню/загальну вартість і визначення найдорожчого пристрою.

3. Структура проєкту та створені модулі

```
inventory-service/
  pyproject.toml, README.md, .gitignore
  src/computer_inventory/
    models.py, services.py, main.py
    analytics.py, streaming.py, domain.py
    fileio.py, database.py
    api.py, async_client.py, performance.py
  src/inventory_service/  # фінальний пакет запуску
  tests/
  scripts/
  docs/GITHUB.md
```

Фінальне дерево також містить Dockerfile, .dockerignore, compose.yaml, .env.example, MANIFEST.in, requirements.lock, migrations та .github/workflows/ci.yml. Пакет inventory_service надає фінальні entry points, а computer_inventory зберігає реалізоване ядро попередніх робіт. Версія — 1.0.0.

4. Метадані, wheel та sdist

```
[build-system]
requires = ["setuptools>=70"]
build-backend = "setuptools.build_meta"

[project]
name = "inventory-service"
version = "1.0.0"
description = "Persistent computer equipment inventory with REST API, variant 13"
authors = [{name = "Капарник Дмитро Андрійович"}]
readme = "README.md"
requires-python = ">=3.11"
dependencies = ["PyYAML>=6.0,<7", "SQLAlchemy>=2.0,<3", "fastapi>=0.115,<1",
"httpx>=0.28,<1", "uvicorn>=0.30,<1", "numpy>=2,<3"]
```

```
[project.scripts]
computer-inventory = "computer_inventory.main:main"
inventory-service = "inventory_service.main:main"

[tool.setuptools.packages.find]
where = ["src"]

[project.optional-dependencies]
test = ["pytest>=8,<10", "pytest-cov>=5,<8"]
dev = ["ruff>=0.12,<1", "mypy>=1.11,<3", "types-PyYAML", "build>=1,<2"]

[tool.pytest.ini_options]
testpaths = ["tests"]

[tool.ruff]
line-length = 88
target-version = "py311"

[tool.ruff.lint]
select = ["E4", "E7", "E9", "F", "I"]

[tool.mypy]
python_version = "3.11"
check_untyped_defs = true
mypy_path = "src"

[tool.setuptools.package-data]
computer_inventory = ["py.typed"]
inventory_service = ["py.typed"]
```

Wheel має універсальний Python-тег py3-none-any, а залежності встановлюються pip окремо для платформи. MANIFEST.in включає потрібні дані та SQL у sdist. У цьому середовищі збірку виконано python -m build --no-isolation із установленими setuptools/wheel; README та CI використовують звичайну ізольовану python -m build.

```
adding 'inventory_service-1.0.0.dist-info/WHEEL'
adding 'inventory_service-1.0.0.dist-info/entry_points.txt'
adding 'inventory_service-1.0.0.dist-info/top_level.txt'
adding 'inventory_service-1.0.0.dist-info/RECORD'
removing build\bdist.win-amd64\wheel
Successfully built inventory_service-1.0.0.tar.gz and inventory_service-1.0.0-py3-none-
any.whl
```

Артефакт	**Результат**
inventory_service-1.0.0-py3-none-any.whl	Побудовано
inventory_service-1.0.0.tar.gz	Побудовано
Встановлення wheel в нове venv	Успішно
Запуск module --demo з wheel	Успішно
Імпорт API factory з wheel	Успішно

```
Version: 1.0.0
Installed wheel import: OK
{"status": "ok"}
```

5. Environment configuration і secrets

```
DATABASE_URL=sqlite:///inventory.db
APP_ENV=development
LOG_LEVEL=INFO
```

```
"""Validated environment configuration; no embedded credentials."""

import os
from dataclasses import dataclass


@dataclass(frozen=True)
class Settings:
    database_url: str = "sqlite:///inventory.db"
    app_env: str = "development"
    log_level: str = "INFO"

    @classmethod
    def from_env(cls) -> "Settings":
        result = cls(
            os.getenv("DATABASE_URL", "sqlite:///inventory.db"),
            os.getenv("APP_ENV", os.getenv("ENVIRONMENT", "development")),
            os.getenv("LOG_LEVEL", "INFO").upper(),
        )
        if result.app_env not in {"development", "test", "production"}:
            raise ValueError("APP_ENV must be development, test or production")
        if result.log_level not in {"DEBUG", "INFO", "WARNING", "ERROR", "CRITICAL"}:
            raise ValueError("Unsupported LOG_LEVEL")
        if not result.database_url.startswith("sqlite:///"):
            raise ValueError("This version supports SQLite database URLs")
        return result
```

DATABASE_URL задає SQLite, APP_ENV (або ENVIRONMENT) — development/test/production, LOG_LEVEL — дозволений рівень. Значення перевіряються до створення API. .env.example містить безпечні приклади, .env і файли БД виключено з Git. Файл .env самостійно не завантажується; Docker може отримати його через --env-file. Секрети не потрібні для локальної SQLite і не вбудовані в код.

6. Docker image, container та persistent volume

```
FROM python:3.12-slim AS builder
WORKDIR /build
COPY pyproject.toml README.md MANIFEST.in ./
COPY src ./src
COPY migrations ./migrations
COPY data ./data
```

```
COPY config.yaml ./
COPY requirements.lock ./
RUN pip install --no-cache-dir build && python -m build

FROM python:3.12-slim AS runtime
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1
DATABASE_URL=sqlite:////data/inventory.db APP_ENV=production LOG_LEVEL=INFO
WORKDIR /app
COPY --from=builder /build/dist/*.whl /tmp/
COPY requirements.lock /tmp/requirements.lock
RUN pip install --no-cache-dir -c /tmp/requirements.lock /tmp/*.whl     && useradd --
create-home --uid 10001 appuser     && mkdir -p /data && chown appuser:appuser /data
USER appuser
VOLUME ["/data"]
EXPOSE 8000
HEALTHCHECK --interval=30s --timeout=3s --start-period=10s --retries=3     CMD python -c
"import urllib.request; urllib.request.urlopen('http://127.0.0.1:8000/health', timeout=2)"
CMD ["uvicorn", "inventory_service.api:create_app", "--factory", "--host", "0.0.0.0", "--
port", "8000"]


.git
.venv
__pycache__
*.pyc
.pytest_cache
.mypy_cache
.ruff_cache
.coverage
htmlcov
.env
dist
build
*.egg-info
*.db
reports


docker build -t inventory-service:1.0.0 .
docker run --name inventory -p 8000:8000 -v inventory-data:/data -e APP_ENV=production -e
DATABASE_URL=sqlite:////data/inventory.db inventory-service:1.0.0
```

Multi-stage build залишає у runtime встановлений wheel. Non-root користувач має право запису в /data; volume зберігає SQLite незалежно від життєвого циклу контейнера. HEALTHCHECK перевіряє /health через urllib. Також підготовлено compose.yaml. 18.10.2026 у GitHub Actions успішно виконано docker build, docker run з volume inventory-data:/data та HTTP-перевірку /health. Docker job завершився успішно за 28 с. Контейнер повернув {"status":"ok"}. Це перевірка на Linux runner GitHub; локальне встановлення Docker Desktop у Windows окремо не підтверджено.

7. Живий API та health

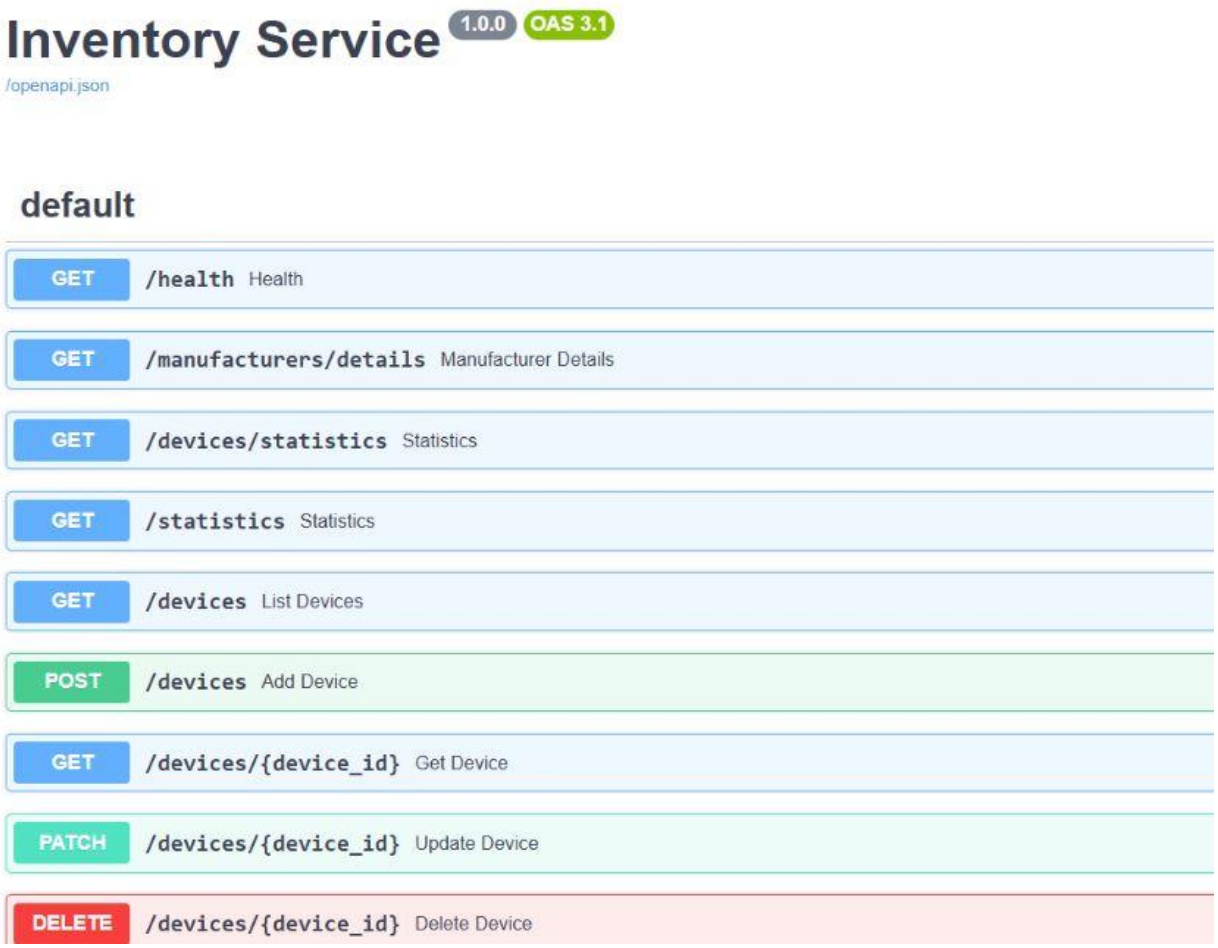


Рисунок 1 — Swagger UI локально запущеного фінального API

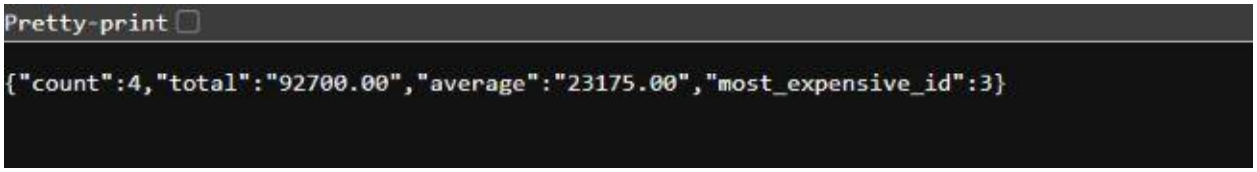


Рисунок 2 — Фактичні підсумки обладнання у відповіді API

```
GET /health
HTTP 200
{"status":"ok"}
```

Цей health-результат отримано від локального Uvicorn, а не від Docker-контейнера. Перевірка health також виконує SELECT 1 до БД. HTTP-контракт та persistence перевірено integration tests і живими запитами.

8. GitHub Actions та локальні quality checks

```
name: Inventory CI
on: [push, pull_request]
jobs:
  quality:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        python-version: ["3.11", "3.12"]
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: ${{ matrix.python-version }}
      - run: python -m pip install -e ".[test,dev]"
      - run: ruff check .
      - run: ruff format --check .
      - run: mypy src
      - run: pytest --cov=computer_inventory --cov=inventory_service --cov-branch --cov-
fail-under=80 --cov-report=term-missing
      - run: python -m build
      - run: python -m venv wheel-check && wheel-check/bin/pip install dist/*.whl &&
wheel-check/bin/python -m inventory_service.main --demo
      - uses: actions/upload-artifact@v4
        with:
          name: packages-${{ matrix.python-version }}
          path: dist/
  docker:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: docker build -t inventory-service:test .
      - run: docker run -d --name inventory -p 8000:8000 -v inventory-data:/data -e
APP_ENV=production inventory-service:test
      - run: |
          for i in $(seq 1 30); do
            if curl --fail http://127.0.0.1:8000/health; then exit 0; fi
            sleep 1
          done
          docker logs inventory
          exit 1
```

Workflow запускається при push і pull_request. Quality matrix перевіряє Python 3.11 та 3.12: Ruff, format check, mypy, pytest із branch coverage й порогом 80%, package build та smoke-test установленого wheel. Окремий job збирає Docker, запускає контейнер і очікує /health. Артефакти пакетів завантажуються через upload-artifact. Фактичний запуск Inventory CI №1 завершився зі статусом Success за 51 с: quality (3.11), quality (3.12) і docker пройшли. Перевірено commit 2e3fb3d3e73f29ffa93689d631746e20beba7caf; створено артефакти packages-3.11 та packages-3.12.

```
All checks passed!


37 files already formatted
```

```
Success: no issues found in 16 source files


.....................................................................................
[100%]
============================== warnings summary ===============================
.venv\Lib\site-packages\fastapi\testclient.py:1
  C:\Users\user\Documents\Codex\2026-10-18\new-chat-2\outputs\computer-
inventory\.venv\Lib\site-packages\fastapi\testclient.py:1: StarletteDeprecationWarning:
Using `httpx` with `starlette.testclient` is deprecated; install `httpx2` instead.
    from starlette.testclient import TestClient as TestClient  # noqa

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
============================== tests coverage ================================
_____________ coverage: platform win32, python 3.12.14-final-0 _____________

Name                                        Stmts   Miss Branch BrPart  Cover   Missing
---------------------------------------------------------------------------------------
src\computer_inventory\__init__.py              1      0      0      0   100%
src\computer_inventory\analytics.py            52      0      8      0   100%
src\computer_inventory\api.py                 145      0     12      0   100%
src\computer_inventory\async_client.py         31      3     12      3    86%   31, 42, 50
src\computer_inventory\database.py            109      2     16      4    95%   65, 67->70,
70->exit, 98
src\computer_inventory\domain.py              106      0     16      1    99%   119->exit
src\computer_inventory\fileio.py              139      1     34      1    99%   104
src\computer_inventory\main.py                122     12     36      7    88%   34-35, 57,
112-113, 115, 119-120, 124, 195, 197, 201
src\computer_inventory\models.py               19      0      8      0   100%
src\computer_inventory\performance.py          78      0     16      0   100%
src\computer_inventory\services.py             25      0      6      0   100%
src\computer_inventory\streaming.py            75      0     24      1    99%   118->113
src\inventory_service\__init__.py               1      0      0      0   100%
src\inventory_service\api.py                    8      0      0      0   100%
src\inventory_service\config.py                17      0      6      0   100%
src\inventory_service\main.py                   3      3      2      0     0%   3-6
---------------------------------------------------------------------------------------
TOTAL                                         931     21    196     17    96%
Coverage HTML written to dir C:\Users\user\Documents\Codex\2026-10-18\new-chat-
2\work\coverage-final
Coverage JSON written to file C:\Users\user\Documents\Codex\2026-10-18\new-chat-
2\work\coverage-final.json
Required test coverage of 80% reached. Total coverage: 96.45%
84 passed, 1 warning, 9 subtests passed in 5.06s
```

9. README, Git та сценарій захисту

README описує встановлення, CLI-режими, змінні середовища, REST routes, дані, тести, benchmark, Docker, CI та демонстрацію. docs/GITHUB.md містить порядок створення порожнього репозиторію, підключення origin, push main і тегів. Підготовлені коміти мають службового автора Lab Preparation; автор майбутніх змін задається власними локальними git config.

```
git log --oneline
git tag
```

git remote -v # origin: https://github.com/dimakaparnyk/professional-python-labs.git

```
git push -u origin main
git push origin --tags
```

На захисті: показати архітектуру й історію; запустити --demo; продемонструвати імпорт і atomic failure; виконати pytest; показати БД та міграцію; виконати CRUD у Swagger; пояснити профіль і benchmark; зібрати пакети; запустити Docker; відкрити реальний workflow у Actions.

10. Production checklist

Перевірка	**Стан**
CLI / предметні операції	Локально перевірено
pytest / branch coverage	84 passed; 96,45%
Ruff check / format	Пройдено
Mypy	16 source-файлів, помилок немає
Wheel / sdist	Побудовано
Wheel у новому venv	Встановлено, demo та імпорт API успішні
Environment configuration / .env.example	Реалізовано, тести пройшли
SQLite / API / health	Локально перевірено
Dockerfile / compose / healthcheck	Build, run та HTTP /health успішні у GitHub Actions
GitHub Actions workflow	Inventory CI №1: Success, 51 с
Screenshots image/container/green CI	Додано скриншот реального успішного pipeline

Підтвердження запуску Docker та CI

Репозиторій: https://github.com/dimakaparnyk/professional-python-labs

Фактичний запуск: https://github.com/dimakaparnyk/professional-python-labs/actions/runs/39135119445

GitHub повідомив про застарілий Node.js 20 у використаних версіях actions та майбутню зміну ubuntu-latest. Ці повідомлення не спричинили помилок перевіреного запуску.

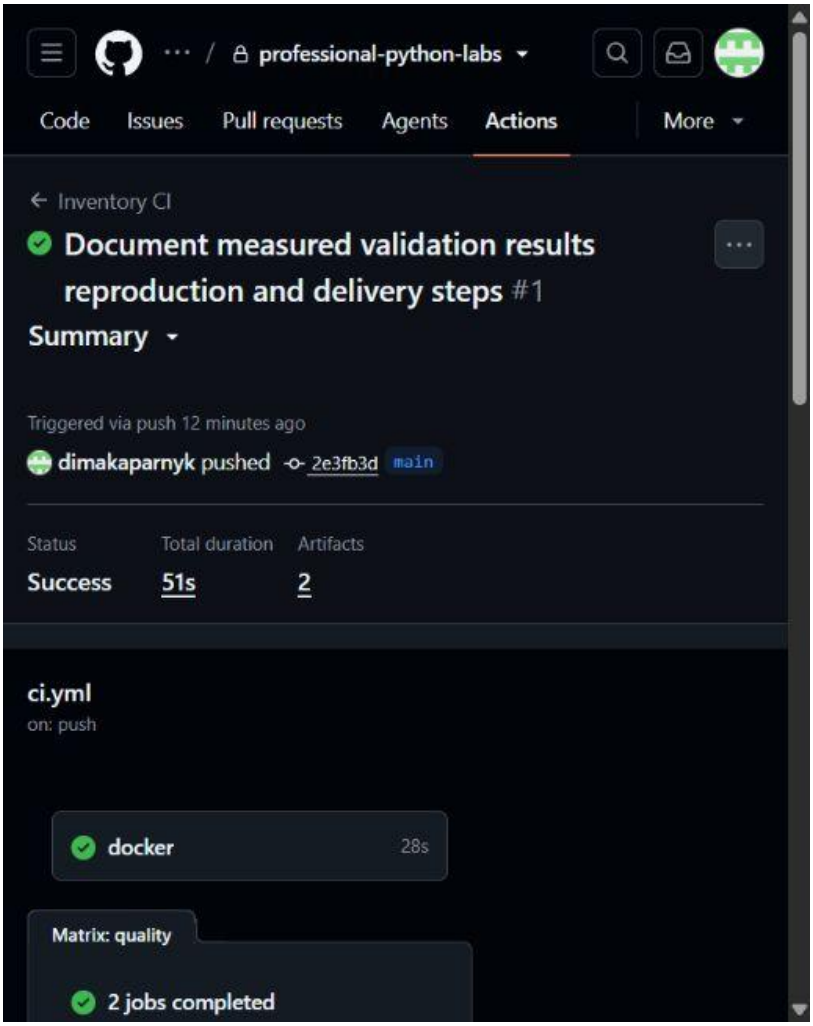


Рисунок 3 — Успішний Inventory CI №1: Docker і дві версії Python

Код і історія лабораторних: https://github.com/dimakaparnyk/professional-python-labs. Для відповідного етапу використовується тег lab-10, фінальна інтегрована реалізація — main (перевірений CI commit 2e3fb3d).

11. Відповіді на контрольні запитання

1. Що таке Python package?

Імпортований модуль або сукупність модулів у просторі імен; звичайний пакет має __init__.py.

2. Що таке distribution package?

Артефакт розповсюдження, який установлює pip і який може містити один або кілька імпортованих пакетів.

3. Що таке wheel?

Готовий до встановлення формат distribution з метаданими й файлами пакета.

4. Яке розширення має wheel?

.whl.

5. Що таке sdist?

Архів вихідного коду для подальшої збірки, зазвичай .tar.gz.

6. Для чого використовується pyproject.toml?

Для метаданих distribution, build backend і конфігурації інструментів проєкту.

7. Що містить [build-system]?

Залежності збірки та ім’я backend, наприклад setuptools.build_meta.

8. Що містить [project]?

Назву, версію, опис, вимоги Python, залежності, автора та інші метадані.

9. Як створити wheel?

Командою python -m build --wheel або повною python -m build.

10. Як створити sdist?

Командою python -m build --sdist або повною python -m build.

11. Для чого використовується python -m build?

Для стандартного виклику build backend та створення wheel і sdist.

12. Що таке semantic versioning?

Схема MAJOR.MINOR.PATCH, що пов’язує зміну версії з характером змін публічного контракту.

13. Що означає MAJOR?

Версію несумісних змін публічного інтерфейсу.

14. Що означає MINOR?

Версію сумісно доданої функціональності.

15. Що означає PATCH?

Версію сумісних виправлень помилок.

16. Що таке environment variable?

Іменоване значення, яке процес отримує від середовища запуску.

17. Навіщо configuration виносити з коду?

Щоб один артефакт можна було налаштовувати для різних середовищ без зміни джерел.

18. Що таке secret?

Конфіденційне значення, яке дає доступ або має інші вимоги до нерозголошення.

19. Наведіть приклади secrets.

Паролі, токени доступу, API-ключі та приватні ключі.

20. Чому secrets не можна commit у Git?

Історія Git зберігає попередні версії та може поширюватися; видалення з поточного файла не усуває витік.

21. Для чого використовується .env?

Для локального зберігання змінних середовища; його завантаження потребує відповідного механізму.

22. Чому .env має бути у .gitignore?

Щоб локальні конфіденційні параметри випадково не потрапили в історію.

23. Для чого потрібний .env.example?

Для документування назв і безпечних прикладів параметрів без реальних секретів.

24. Що таке Docker?

Платформа збирання та запуску ізольованих контейнерів на основі образів.

25. Що таке Dockerfile?

Файл інструкцій побудови образу.

26. Що таке Docker image?

Шаблон файлової системи та метаданих запуску контейнера.

27. Що таке Docker container?

Запущений або зупинений екземпляр образу з власним станом виконання.

28. Чим image відрізняється від container?

Образ є шаблоном; контейнер — створеним із нього екземпляром із процесами та змінюваним шаром.

29. Для чого використовується docker build?

Для створення образу за Dockerfile та build context.

30. Для чого використовується docker run?

Для створення й запуску контейнера з образу та заданими параметрами.

31. Що робить EXPOSE?

Документує порт застосунку в образі; сам по собі не публікує його на хості.

32. Що робить CMD?

Задає типову команду запуску контейнера.

33. Для чого потрібний WORKDIR?

Задає робочий каталог для наступних інструкцій і типової команди.

34. Що робить COPY?

Копіює файли з build context або іншого етапу в образ.

35. Для чого використовується .dockerignore?

Щоб не передавати у build context зайві файли, середовища, кеші та секрети.

36. Що таке Docker volume?

Кероване Docker сховище даних, життєвий цикл якого не залежить від конкретного контейнера.

37. Для чого потрібний health endpoint?

Для перевірки доступності застосунку та потрібних залежностей, наприклад БД.

38. Що таке CI?

Continuous Integration — автоматична перевірка змін після інтеграції в репозиторій.

39. Що таке CD?

Continuous Delivery або Deployment — підготовка до випуску чи автоматичне розгортання залежно від процесу.

40. Що таке GitHub Actions?

Сервіс GitHub для запуску workflow за подіями репозиторію.

41. Що таке workflow?

YAML-опис подій, jobs, steps і параметрів автоматизації.

42. Що таке job?

Набір steps, що виконується на одному runner.

43. Що таке step?

Окрема дія або команда всередині job.

44. Що таке runner?

Середовище, яке виконує job, наприклад Ubuntu-машина GitHub.

45. Коли повинен запускатися CI?

При push і pull_request, а за потреби вручну, за тегом або розкладом.

46. Для чого потрібний lint?

Для виявлення помилок і порушень стилю до запуску чи рев'ю.

47. Для чого використовується Ruff?

Для швидкого lint і перевірки або застосування форматування Python-коду.

48. Для чого потрібний mypy?

Для статичного виявлення несумісності типів у коді.

49. Що таке type checking?

Аналіз відповідності значень і операцій оголошеним типам.

50. Навіщо tests запускати в CI?

Щоб кожна зміна проходила однакові незалежні перевірки та не вносила регресій.

51. Навіщо package build виконувати в CI?

Щоб виявити помилки метаданих, відсутні файли й непрацездатний distribution.

52. Навіщо Docker build виконувати у CI?

Щоб виявити проблеми Dockerfile, залежностей і контексту до розгортання.

53. Що таке GitHub Secret?

Захищене значення конфігурації репозиторію або середовища для workflow.

54. Як використовувати secret у workflow?

Через контекст secrets.NAME у потрібному step; значення не вбудовують у код або журнали.

55. Що таке documentation?

Опис контрактів, архітектури, встановлення, використання та супроводу системи.

56. Що повинен містити README?

Призначення, вимоги, встановлення, конфігурацію, запуск, API, тести, Docker, CI й приклад демонстрації.

57. Для чого потрібні docstrings?

Для пояснення публічних функцій, параметрів, результатів та винятків без звернення до реалізації.

58. Що таке OpenAPI?

Машинозчитуваний опис HTTP API з маршрутами, параметрами та схемами.

59. Що таке /docs у FastAPI?

Інтерактивна Swagger UI-документація за типовими налаштуваннями FastAPI.

60. Що таке /openapi.json?

JSON-опис OpenAPI-контракту застосунку.

61. Що таке logging?

Запис діагностичних і робочих подій з рівнями, часом та контекстом.

62. Чому logging краще за print?

Підтримує керовані рівні, формати й обробники, потрібні для діагностики розгорнутого застосунку.

63. Що таке production configuration?

Перевірені параметри робочого середовища, доступу до даних, журналювання та запуску.

64. Чим development відрізняється від production?

Development оптимізує зручність змін, production — відтворюваність, контроль конфігурації та надійний запуск.

65. Що таке Docker healthcheck?

Періодична команда, яка визначає стан healthy або unhealthy контейнера.

66. Для чого потрібний non-root user?

Щоб зменшити наслідки помилки або компрометації процесу всередині контейнера.

67. Що таке multi-stage build?

Dockerfile з кількома етапами, де фінальний образ отримує лише потрібні артефакти.

68. Що таке coverage?

Метрика виконаних під час тестів рядків або гілок коду.

69. Навіщо встановлювати coverage threshold?

Щоб автоматично виявляти падіння перевіреного обсягу, пам’ятаючи, що відсоток не гарантує правильність.

70. Що означає green CI pipeline?

Що всі обов’язкові jobs конкретного запуску workflow успішно завершилися.

12. Висновок

Сформовано фінальний Python distribution із конфігурацією через середовище, предметними функціями, SQLite та API. Пройдено локальні тести, lint, format, type checking, збірку й перевірку wheel. Реальний GitHub Actions pipeline підтвердив якість на Python 3.11/3.12, побудову Docker-образу, запуск контейнера та доступність /health.